

Intermediate (timeouts and retry-with-backoff) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `retry_backoff_practice.py` (Intermediate section): a network call with no timeout can hang forever, and a network call that just retries instantly on failure makes a busy server's day worse, not better. This exercise builds the retry-with-backoff loop that becomes `http_client.py`'s core almost unchanged. **If not:** the Build Task's retry logic would be written from scratch under deadline pressure, with no smaller, already-working version to copy from — exactly the mistake this document's whole structure is built to avoid.

Basic Version

Approach 1 — a plain retry loop

```
# retry_backoff_practice.py – Intermediate section
import time
import requests

# part 1: prove the timeout fires
try:
    requests.get("http://10.255.255.1", timeout=(1, 2))
except requests.exceptions.Timeout:
    print("timed out, as expected")

# part 2: retry loop with backoff
max_attempts = 5
response = None

for attempt in range(max_attempts):
    try:
        response = requests.get("https://api.github.com", timeout=(3, 5))
        break
    except requests.exceptions.Timeout:
        print("attempt", attempt, "timed out")
        if attempt == max_attempts - 1:
            print("gave up after", max_attempts, "attempts")
        else:
```

```
        time.sleep(2 ** attempt)

    if response is not None:
        print(response.status_code)
```

Expected output (the real call succeeds on attempt 0, since `api.github.com` responds fine):

```
timed out, as expected
200
```

This works. It's missing type hints and re-raises nothing — a caller of this script-level code can't distinguish "it worked" from "it gave up" except by checking `response is not None` themselves.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a typed, reusable function that re-raises

```
# retry_backoff_practice.py – Intermediate section
import logging
import time
import requests

logger = logging.getLogger(__name__)

def get_with_retry(url: str, max_attempts: int = 5) -> requests.Response:
    for attempt in range(max_attempts):
        try:
            return requests.get(url, timeout=(3, 5))
        except requests.exceptions.Timeout:
            if attempt == max_attempts - 1:
                raise
            wait_seconds = 2 ** attempt
            logger.warning(
                "attempt %s timed out, waiting %ss", attempt, wait_seconds
            )
            time.sleep(wait_seconds)
    raise RuntimeError("unreachable") # loop always returns or raises above

if __name__ == "__main__":
    try:
        requests.get("http://10.255.255.1", timeout=(1, 2))
    except requests.exceptions.Timeout:
        print("timed out, as expected")

    response = get_with_retry("https://api.github.com")
    print(response.status_code)
```

Expected output:

```
timed out, as expected
```

200

Difference from Basic: pulling the loop into `get_with_retry()` means a caller gets back either a real `Response` or a real, propagated `requests.exceptions.Timeout` — never a silent `None` they have to remember to check. Type hints (`url: str, -> requests.Response`) also turn the function signature into documentation. The retry notice also moves from `print()` to `logger.warning()` (Doc01's `logging_setup.py` pattern) — this is a diagnostic about the program's own internal state, not output the reader is meant to see, so it belongs on the logger, not `stdout`.

Approach 2 — jitter, and the backoff math split into its own testable function

```
# retry_backoff_practice.py – Intermediate section
import random
import time
import requests

def compute_backoff_delay(attempt: int) -> float:
    """Exponential backoff with jitter: 2**attempt seconds, plus a small
    random extra so many failing callers don't retry in lockstep."""
    # why: pulled out on its own so it's testable with a plain assertion –
    # no network, no time.sleep, no mocking needed.
    # how: 2**attempt doubles the wait each retry (1, 2, 4, 8...); the
    # random.uniform(0, 1) jitter stops many clients retrying at the exact same
    # instant.
    return (2 ** attempt) + random.uniform(0, 1)

def get_with_retry(url: str, max_attempts: int = 5) -> requests.Response:
    # why: gives the caller back a real Response or a real, propagated
    # error – never a silent None they have to remember to check.
    # how: remembers the last failure so it can be attached to the final raise
    last_error: Exception | None = None

    for attempt in range(max_attempts):
        try:
            # when: success returns immediately, no further attempts
            return requests.get(url, timeout=(3, 5))
        except requests.exceptions.Timeout as e:
            last_error = e
            if attempt < max_attempts - 1:
                # when: only sleep if there's another attempt coming –
                # no point waiting after the very last try.
                time.sleep(compute_backoff_delay(attempt))
```

```
        raise TimeoutError(f"gave up after {max_attempts} attempts") from last_error

if __name__ == "__main__":
    # roughly 8.something, no network call needed
    print(compute_backoff_delay(3))
    response = get_with_retry("https://api.github.com")
    print(response.status_code)
```

Expected output:

```
8.417...
200
```

The first line proves `compute_backoff_delay(3)` works entirely on its own, with no server, no `time.sleep`, and no mocking — that's the whole point of separating it from the loop. A well-tested library, `urllib3.util.Retry` (the library `requests` itself is built on), can do this same job with no loop in your own code at all — worth knowing exists, but it doesn't distinguish "worth retrying" as flexibly as your own function can, which is exactly what this document's Build Task needs.

Difference from Approach 1: Approach 1's loop is correct but retries every failing caller at exactly the same moments, and its backoff math is buried inside the loop where it can't be tested without a real (or mocked) network call. Approach 2 fixes both: jitter breaks the lockstep-retry problem, and `compute_backoff_delay()` becomes a pure function you can test with a plain assertion.

Which one should you actually write? For this document's Build Task, Approach 2's pattern — a small, pure `compute_backoff_delay(attempt)` function, separate from the loop that touches the network — is what you want, because the Build Task's constraint is explicit: your retry code needs to be testable without making real network calls.